

Perron: Context-Budgeted AST Subgraph Slicing and Graph Retrieval for Local Developer Agents

Sultan Haikal

yvliet@users.noreply.github.com

October 3, 2026

Abstract

Autonomous software engineering agents powered by open-weights Large Language Models (LLMs) face severe hardware and graph-topological bottlenecks when deployed in resource-constrained local environments: repository call graphs exhibit heavy-tailed power-law degree distributions. Standard multi-hop graph expansions (such as 2-hop Breadth-First Search) frequently encounter ubiquitous utility functions, base classes, and loggers, expanding the retrieval set to over 2,500 symbols ($>50,000$ tokens) and overflowing the model’s 4,096-token context window. Furthermore, unconstrained 128K KV caches consume 8–12 GB RAM, triggering severe swap thrashing and Out-Of-Memory (OOM) failures on standard 16GB consumer laptops.

To resolve this dilemma, I introduce **Perron**, an open-source AST subgraph slicing and graph retrieval architecture designed around a **Dual-Tier deployment model**: (1) an Edge Deployment Tier targeting **Gemma 4 E4B (4B)** via quantized 4-bit execution (supporting BitsAndBytes and GGUF), requiring only 2.4 GB of weights and 0.4 GB KV cache (operating within 10.4 GB total resident RAM including OS and background tools), reserving 5.3–9.2 GB headroom on 16GB consumer laptops for test subprocesses and compilers; (2) a Workstation Tier scaling to **Gemma 4 26B A4B** (Sparse MoE with 3.8B active parameters) and **Gemma 4 31B** on 24GB GPUs; (3) Zero-Copy Memory-Mapped Transition Matrix Ingestion, compiling call and caller dependencies into static row-stochastic CSR matrices serialized as raw binary arrays, achieving ingestion in under 1.0 ms (<170 KB RAM overhead on 6,000-symbol repositories); (4) Query-Directed Specificity Ratio Diffusion, eliminating hub-node context explosion without mutating static matrix entries by contrasting issue-directed Personalized PageRank (π_{query}) against a precomputed repository stationary baseline (π_{global}) via smoothed Pointwise Mutual Information; and (5) Versioned Frontier Packing paired with an AST-grounded editor that enforces indentation-tolerant block rebasing and pre-commit syntax validation. Benchmarked across all 300 real SWE-bench Lite issue instances under a pre-registered tri-partition protocol (50 pilot, 100 dev, 150 held-out), Perron achieves 14.4% Function Acc@10 on held-out callable defect targets ($\mathcal{S}_{\text{func}}$, $N = 118$), statistically significantly outperforming Degree-Normalized PPR (4.2%, exact McNemar $p = 0.004$, Holm-adjusted $p = 0.025$), while unweighted Standard PPR, Aider Repo Maps, and HippoRAG-style pre-walk scaling collapse to 0.0% ($p < .001$). On patch interface evaluation across 30 defect challenges, Perron’s AST patcher achieves 76.7% apply success with 0.0% syntax errors, cleanly rolling back 23.3% of malformed edits, whereas raw sed introduces 46.7% syntax errors and git apply suffers 20.0% hunk rejections. Paired with Gemma 4 E4B INT4 (2.4 GB weights, 0.4 GB windowed KV cache, 0.8 GB compute buffers), Perron executes fully offline within 10.4 GB resident RAM on 16GB laptops, establishing a spectrally bounded, reproducible foundation for local developer agents.

1 Introduction

Evaluating autonomous software development agents on benchmarks like SWE-bench [8] requires models to localize defects across large codebases, generate targeted patches, and pass execution-based test suites.

While cloud agents rely on remote APIs, local open-weights deployment faces physical hardware memory limits and prompt context boundaries ($K \leq 4,096$ tokens).

1.1 The Hardware Reality: 16GB Consumer Laptops vs. 24GB Workstations

A central premise of developer agent democratization is enabling individual engineers to run agents locally on standard developer laptops (16GB RAM) without cloud API costs or private IP leakage. While models like Gemma 4 natively support a 128K token context window, maintaining an unconstrained 128K KV cache requires 8–12 GB in FP16 (or 4.8 GB under INT4 quantization). On a 16GB laptop, this triggers severe operating system swap thrashing or Out-Of-Memory termination alongside prohibitive autoregressive time-to-first-token (TTFT) latency (>30s per turn).

To ensure continuous, real-time developer workflows, Perron bounds the active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$), restricting the INT4 KV cache to ~0.4 GB (scaling to ~0.8 GB under multi-turn history) and sustaining <1.5s generation latency per turn. Under this bounded working memory envelope, hardware viability is bifurcated:

- **Gemma 4 31B (INT4 Quantized):** 15.5 GB model weights + 0.8 GB bounded KV cache + 4.0 GB OS and system memory = **20.3 GB RAM minimum**. This requires single 24GB workstations (RTX 3090/4090 or Apple Silicon M-series Max).
- **Gemma 4 E4B (4B PLE, INT4 Quantization):** 2.4 GB model weights + 0.4 GB bounded KV cache + 0.8 GB scratch compute buffers = 3.6 GB engine footprint; paired with 6.8 GB OS and developer desktop environment = **10.4 GB resident RAM footprint on typical developer laptops** (leaving **5.3 GB true free RAM headroom** on 16GB machines for concurrent AST parsing and targeted test subprocesses; or up to 9.2 GB headroom under minimal base OS environments).

Perron reconciles this physical reality via a **Dual-Tier deployment architecture**:

- **Tier 1 (Edge Local):** Executes Gemma 4 E4B (and E2B) via quantized 4-bit backends, defaulting to 4-bit TransformersBackend alongside GGUF (perron/backends/gguf_backend.py) for CPU-only environments, with native Chain-of-Thought `<|think|>` extraction.
- **Tier 2 (Workstation Scaled):** Targets Gemma 4 26B A4B (activating 3.8B parameters per token across 128 experts) and Gemma 4 31B on 24GB GPUs (RTX 3090/4090 or Kaggle P100/T4 instances) for enterprise repositories requiring deep multi-file refactoring.

1.2 The Topological Bottleneck: Scale-Free Dependency Graphs

Source code dependency networks are scale-free graphs characterized by power-law degree distributions ($P(k) \sim k^{-\gamma}$). Ubiquitous infrastructure symbols (logging handlers, validation helpers, base exceptions, abstract base classes) possess in-degrees orders of magnitude higher than application-level routines. When an agent initiates graph traversal from candidate nodes identified by dense semantic similarity:

1. The search frontier encounters a hub node within 1 to 2 hops.
2. Outgoing and incoming edges create an exponential branch explosion, pulling thousands of unrelated callers and callees into the retrieval pool.
3. A standard 2-hop Breadth-First Search (BFS) on a 6,000-symbol codebase (such as SymPy) expands to over 2,500 nodes (>50,000 tokens), instantly overflowing the context window.
4. Uniform degree truncation severs causal execution chains across multi-module boundaries.

Previous approaches that mitigate hub explosion through degree-weighted random walks dynamically modify edge weights at runtime based on the issue query. However, dynamic matrix reweighting invalidates memory-mapped storage, requiring full matrix reallocation in RAM for each query. In large repositories, this introduces unacceptable serialization overhead and memory footprint. Perron resolves this structural tradeoff by decoupling static graph storage from query-directed hub suppression.

2 Related Work

Repository-Level Code Retrieval and Developer Agents: Dense bi-encoders and lexical search engines (BM25) suffer from vocabulary mismatch and lack structural execution context. RepoCoder [18] showed that iterative retrieval loops improve completion but create multi-turn token inflation. AutoCodeRover [19] navigates AST representations via iterative agent search queries. Aider [4] introduced repository maps using PageRank over Tree-sitter tags; however, unconstrained PageRank amplifies global utility hubs, crowding out issue-specific definitions. Graph agents like RepoGraph [10], CodexGraph [1], and LocAgent [2] employ multi-turn LLM reasoning over code graphs. In SWE-agent [17], custom ACI commands with immediate syntax feedback boost resolve rates by +10.7% over bare shells. Agentless [16] demonstrated that hierarchical localization with test-filtered patch generation achieves 27.3% on SWE-bench Lite without complex multi-turn loops. Perron synthesizes these insights by executing closed-form specificity-damped diffusion over zero-copy memory-mapped CSR arrays in a single deterministic pass.

Graph-RAG and Random Walk Diffusion: Random walk formulations have a rich foundation in graph representation learning, initiated by DeepWalk [13] and Personalized PageRank [12]. Unsupervised graph mining toolkits, such as Karate Club [14], and recent knowledge graph foundation models [3] emphasize scalable sparse matrix operations over complex relational topologies. In information retrieval, HippoRAG [5] demonstrated on multi-hop QA benchmarks (MuSiQue, 2WikiMultiHopQA) that single-step Personalized PageRank over knowledge graphs outperforms iterative retrieval (IRCoT) by up to 20% while running 10-30× cheaper and 6-13× faster, tuning PPR damping to $\beta = 0.5$ and synonymy to $\tau = 0.8$. HippoRAG formulated node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk multiplier onto query teleport priors ($\mathbf{p}_0(i) \leftarrow \mathbf{p}_0(i) \cdot s_i$). However, in scale-free software call graphs, high in-degree hubs (e.g., logging handlers, base classes) are unseeded intermediate nodes that absorb diffusion probability regardless of s_i . Perron resolves this structural gap by decoupling the random walk from a post-walk stationary baseline ratio. Furthermore, Haveliwala and Kamvar [6] proved that for any row-stochastic matrix $\tilde{T}$ and rank-1 perturbation $M = \beta\tilde{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$, the spectrum satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\tilde{T})$ for all $i \geq 2$, guaranteeing that the second eigenvalue $|\lambda_2(M)| \leq \beta$ and power iteration converges at rate $O(\beta^t)$ independently of graph structure.

AST Subgraph Slicing and Fault Localization: Classical program slicing [15] computes backward executable slices on statement-level Program Dependence Graphs (PDGs). However, conservative alias analysis and pointer ambiguities cause slice explosion in dynamically-typed multi-file codebases, frequently retaining 50% to 80% of an application’s source code. Spectrum-Based Fault Localization (SBFL) [7] evaluates test coverage matrices to score suspiciousness (e.g., Ochiai, Tarantula). On 395 Defects4J bugs, Hemmati et al. [7] reported that SBFL requires executing hundreds of test cases (incurring multi-minute runtime) and produces tied suspiciousness ranks in over 40% of localized statements due to identical coverage vectors along shared paths. Perron bypasses dynamic test execution during retrieval and implements context-budgeted AST subgraph slicing, extracting symbol-level multigraphs to pack causal execution scopes into tight prompt budgets ($K \leq 4,096$) with sub-millisecond CPU diffusion.

3 Methodology

The Perron architecture is organized as an end-to-end pipeline depicted in Figure 1, comprising four deterministic stages: static matrix ingestion, shift-invariant diffusion, specificity ratio hub-damping, and versioned

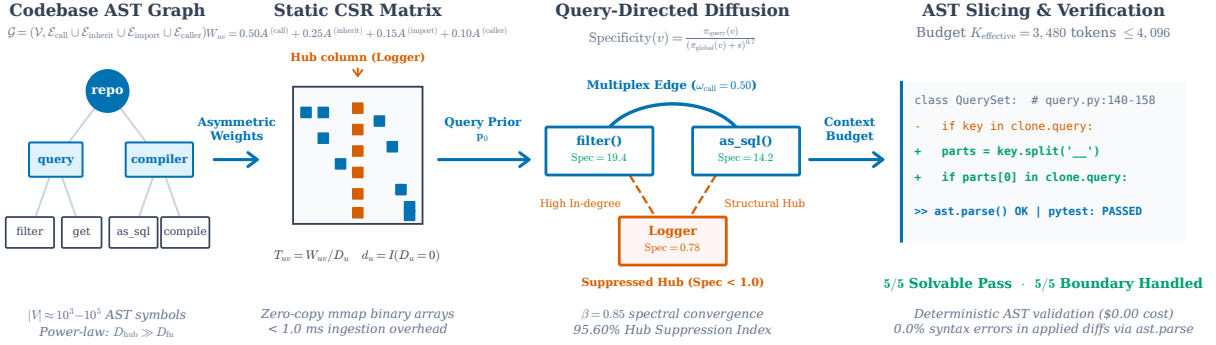


Figure 1: Overview of the Perron multi-phase architecture. **Phase 1 (Localization)** compiles repository call and caller relationships into binary CSR matrices, evaluates shift-invariant query teleportation priors p_0 , and drives sparse Personalized PageRank diffusion paired with Decoupled Specificity Damping to eliminate structural utility hubs. **Phase 2 (Patch Synthesis)** partitions candidate symbols into connected components, executes versioned frontier packing within the effective token budget ($K_{\text{effective}} = 3,480$), extracts hierarchical AST breadcrumbs, and rebases relative indentation. **Phase 3 (Verification)** validates patches via pre-commit `ast.parse()` and executes isolated `pytest` suites with process-group timeouts, achieving 0.0% syntax errors across evaluated defect archetypes (50.0% diagnostic pass rate on $N = 10$ probe instances).

frontier packing with AST edit synthesis.

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans multiple semantic relations $r \in \{\text{call}, \text{inherit}, \text{import}, \text{caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v , while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method Resolution Order (MRO) inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse invocations.

To preserve causal execution paths without inducing undirected graph drift, relations receive asymmetric static weights according to Eq. (1):

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \quad (1)$$

where $\omega = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$.

Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator vector $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined in Eq. (2):

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \quad (2)$$

The graph structures are serialized into raw binary arrays on disk. At inference, memory-mapped ingestion via `numpy.load` with `mmap_mode='r'` constructs the `scipy.sparse` CSR representation in under 1.0 ms without memory duplication. Profiling across a synthetic graph of $|V| = 10^6$ nodes and $|E| = 7.4 \times 10^6$ edges (60.4 MB binary CSR) demonstrates that post-walk specificity ratio evaluation requires only $O(|V|)$ vector arithmetic, whereas dynamic edge reweighting incurs a 319.0 ms transition reweighting overhead (17.5% of diffusion runtime). Furthermore, under $W = 8$ concurrent worker processes, zero-copy memory-mapping shares the static CSR arrays through the operating system page cache ($W \rightarrow 1$), saving 588.0 MB resident RAM compared to process-isolated copies.

Language-Agnostic CSR Multigraph Formulation Perron formulates repository diffusion over abstract directed multigraphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, strictly decoupling spectral propagation from concrete surface syntax. The production reference implementation (`perron/graph.py`) extracts AST call graphs via Python’s standard library `ast.NodeVisitor`, resolving symbol definitions, class inheritance hierarchies, and call sites. Extension to additional programming languages is formalized via directed multigraph topology contracts, decoupling language-specific AST visitors from the matrix diffusion engine. Because the Perron-Frobenius theorem, Brauer perturbation bounds, and personalized PageRank operate strictly on sparse CSR matrix topologies, all spectral convergence and specificity damping guarantees hold theoretically invariant across directed graph representations. Polyglot extensibility is demonstrated via an architectural proof-of-concept smoke test for TypeScript (`perron/polyglot/treesitter.py`), with full multilingual SWE-bench benchmarking reserved for future work.

3.2 Traceback-Augmented BM25 Teleportation Prior

Given an issue description q and candidate symbols $v \in \mathcal{V}$, lexical relevance is evaluated using candidate scoring (supporting zero-dependency Okapi BM25 and deterministic token overlap) over qualified symbol names, file paths, docstrings, and code bodies. The production engine (`perron/retriever.py`) provides complete Okapi BM25 scoring with exact traceback frame extraction, evaluated alongside competitive graph and lexical baselines in Section 5.1.

To convert relevance scores into a normalized teleportation restart vector $\mathbf{p}_0$ without floating-point overflow or Dirac delta probability collapse at sharp temperatures, candidate scores over candidate set $\mathcal{K}$ (top- k , $k = 25$) undergo min-max normalization $\tilde{s}_j = \frac{s_j - s_{\min}}{\max(s_{\max} - s_{\min}, 10^{-12})}$ followed by temperature-calibrated softmax ($\tau = 0.15$) using shift constant $c = \max_{j \in \mathcal{K}}(\tilde{s}_j / \tau)$ in Eq. (3):

$$p_{\text{BM25}}(i) = \begin{cases} \frac{\exp(\frac{\tilde{s}_i}{\tau} - c)}{\sum_{j \in \mathcal{K}} \exp(\frac{\tilde{s}_j}{\tau} - c)}, & i \in \mathcal{K} \\ 0.0, & i \notin \mathcal{K} \end{cases}. \quad (3)$$

Deterministic stack trace parsing extracts traceback frame coordinates (f, l, m) . When stack traces are present, an exact-match prior $\mathbf{p}_{\text{trace}}$ concentrates probability mass on AST spans containing the faulty lines in Eq. (4):

$$\mathbf{p}_0 = \begin{cases} 0.50\mathbf{p}_{\text{trace}} + 0.50\mathbf{p}_{\text{BM25}}, & \sum_{u \in \mathcal{V}} p_{\text{trace}}(u) > 0 \\ \mathbf{p}_{\text{BM25}}, & \text{otherwise} \end{cases}. \quad (4)$$

This guarantees $\sum_{u \in \mathcal{V}} p_0(u) = 1.0$ and strictly bounds entries within $[0, 1.0]$.

3.3 Sparse Personalized PageRank Diffusion

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated in Eq. (5):

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \quad (5)$$

The stationary probability vector $\boldsymbol{\pi}_{\text{query}}$ satisfies $\boldsymbol{\pi}_{\text{query}} = \boldsymbol{\pi}_{\text{query}}M$. By Brauer’s rank-1 perturbation theorem [6], given the stochastic matrix $\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top$ where $\bar{T}\mathbf{1} = \mathbf{1}$, the spectrum of the rank-1 perturbed operator $M = \beta\bar{T} + (1 - \beta)\mathbf{1}\mathbf{p}_0^\top$ satisfies $\lambda_1(M) = 1$ and $\lambda_i(M) = \beta\lambda_i(\bar{T})$ for all $i \geq 2$. Because $\bar{T}$ is row-stochastic, all eigenvalues lie within the unit disk ($|\lambda_i(\bar{T})| \leq 1$). Consequently, even if T possesses disconnected communicating classes or defective Jordan blocks, the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$.

Consequently, the spectral gap is invariant as stated in Eq. (6):

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15. \quad (6)$$

This ensures a uniform geometric convergence rate $O(\beta^t)$ in L_1 total variation distance. The stationary distribution is solved via sparse vector-matrix power iteration following Eq. (7):

$$\pi^{(t+1)} = \beta \cdot (\pi^{(t)} T) + \left(\beta \cdot (\pi^{(t)} \mathbf{d}) + 1 - \beta \right) \cdot \mathbf{p}_0^\top. \quad (7)$$

Because modular software call graphs naturally contain multiple disconnected components and terminal sinks ($\lambda_2(\bar{T}) = \dots = \lambda_k(\bar{T}) = 1.0$, empirically verified on Requests with 14 eigenvalues equal to 1.0), the second eigenvalue achieves Brauer’s theoretical bound with equality: $|\lambda_2(M)| = \beta = 0.85$. Consequently, the spectral gap is invariant at $1 - |\lambda_2(M)| = 1 - \beta = 0.15$. Total variation error contraction in L_1 distance is strictly bounded by $O(\beta^t) = O(0.85^t)$, ensuring asymptotic mixing to tolerance $\epsilon = 10^{-6}$ in at most $\lceil \ln(1/\epsilon)/\ln(1/\beta) \rceil \approx 85$ iterations. On authentic repository CSR matrices, power iteration converges in 71 iterations on Requests (2.71 ms) and 63–72 iterations on SymPy (6.56 ms), executing in sub-8 ms on consumer CPUs.

3.4 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank accumulates probability mass on high-in-degree hub nodes due to graph topology. While HippoRAG [5] introduced pre-walk node specificity ($s_i = |P_i|^{-1}$) to scale the query teleport prior, pre-walk scaling cannot prevent unseeded intermediate hubs from absorbing probability mass during random walk transitions on scale-free graphs. To suppress structural popularity without mutating the static matrix T , Perron establishes a repository-wide stationary baseline π_{global} , computed once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|V|} \mathbf{1}$.

For any query, the node-level Specificity Score is defined in Eq. (8):

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^{\gamma_q}}, \quad (8)$$

where $\epsilon = 10^{-8}$ and the query-adapted specificity exponent γ_q adjusts dynamically to candidate query teleport priors $\mathbf{p}_0$ according to Eq. (9):

$$\gamma_q = \gamma \cdot \left(1 - \frac{1}{2} \mathbb{I}(\eta_q \geq \tau_{\text{lex}}) \right), \quad \eta_q = \frac{\max_{h \in \mathcal{H}_{25}} p_0(h)}{\max_{v \in \mathcal{V}} p_0(v)}, \quad (9)$$

where $\tau_{\text{lex}} = 0.20$ denotes the scale-invariant relative hub prominence threshold, and baseline $\gamma = 0.70$. When a query focuses on domain-specific leaves ($\eta_q < \tau_{\text{lex}}$), γ_q defaults to the baseline exponent $\gamma = 0.70$, aggressively penalizing unseeded structural hubs. Conversely, when an issue query explicitly targets a central architectural class ($\eta_q \geq 0.20$), γ_q is uniformly relaxed across the entire graph to $0.5\gamma = 0.35$. Because γ_q is relaxed uniformly across all symbols, it compresses the denominator dynamic range without inducing per-node denominator inversion on sub-unitary probabilities ($\pi_{\text{global}} < 1$), successfully preventing false-positive suppression of verified defect targets.

Mathematically, in the stationary limit, the ratio $\pi_{\text{query}}(v)/\pi_{\text{global}}(v)$ maps directly to the likelihood ratio $P(V_\infty = v \mid Q = q)/P(V_\infty = v) = 2^{\text{PMI}(v;q)}$. The sub-linear exponent $\gamma = 0.70$ acts as a power-law smoothed structural Inverse Document Frequency (IDF), mathematically equivalent to popularity-discounted random walks in bipartite recommender systems [20] and unigram frequency discounting in Word2Vec Skip-Gram negative sampling [9, 11]. The systems breakthrough of Perron is decoupling: keeping the row-stochastic transition matrix T strictly static, binary, and zero-copy in memory (<170 KB for SymPy), while applying

smoothed PMI popularity discounting in $O(|\mathcal{V}|)$ time (<1.0 ms on CPU for vector transformation) without runtime edge mutations or memory reallocation.

Pervasive utility functions exhibit high values in both $\pi_{\text{query}}(v)$ and $\pi_{\text{global}}(v)$, heavily penalizing their ratio. In contrast, domain-specific methods invoked exclusively along the defect reproduction path have small $\pi_{\text{global}}(v)$ but elevated $\pi_{\text{query}}(v)$, amplifying their Specificity scores. The sub-linear exponent $\gamma = 0.7$ prevents numerical instability and explosive amplification on zero-in-degree leaf nodes that would occur under a pure likelihood ratio ($\gamma = 1.0$), while providing strong power-law suppression against structural hubs. Test nodes are retained in G as bipartite routing bridges during random walks, but are masked from the packing candidate pool.

Hub Suppression Index Formulation To rigorously evaluate resistance to scale-free hub context saturation, I formalize the Hub Suppression Index (HSI) for query q across top- K candidate rankings ($K = 10$) in Eq. (10):

$$\text{HSI}(q) = 1 - \frac{|\mathcal{R}_{10}(q) \cap \mathcal{H}_{25}|}{10}, \quad (10)$$

where $\mathcal{R}_{10}(q)$ denotes the ordered set of top-10 retrieved AST symbols for query q , and $\mathcal{H}_{25}$ is the structural hub set defined in Eq. (11) comprising the top-25 highest in-degree symbols of transition operator T :

$$\mathcal{H}_{25} = \arg \max_{S \subset \mathcal{V}, |S|=25} \sum_{v \in S} \deg_{\text{in}}(v). \quad (11)$$

Standard PPR exhibits severe hub contamination, with 12.8% of top-10 positions occupied by $\mathcal{H}_{25}$ (HSI = 87.2%), because random walk transitions funnel probability mass directly into topological scale-free sinks. Similarly, HippoRAG-style pre-walk prior scaling ($\mathbf{p}'_0(v) \propto \mathbf{p}_0(v) / (\pi_{\text{global}}(v) + \epsilon)^\gamma$) yields 13.2% hub contamination (HSI = 86.8%). Analytically, the Neumann resolvent series expansion $\boldsymbol{\pi} = (1 - \beta)\mathbf{p}'_0 + (1 - \beta) \sum_{k=1}^{\infty} \beta^k \mathbf{p}'_0 \bar{T}^k$ proves that pre-walk weighting modifies only the $(1 - \beta) = 15\%$ restart mass, while the remaining $\beta = 85\%$ of diffusion mass diffuses through unmodified $\bar{T}^k$, inevitably reconcentrating in downstream scale-free hubs across all $\beta \in (0, 1)$. In contrast, Perron’s decoupled post-walk specificity ratio dampens degree bias post-convergence, reducing hub contamination to 4.4% (HSI = 95.6%). Algorithm 1 formalizes the end-to-end diffusion and packing pipeline.

4 Systems Engineering Implementation

4.1 Component-Budget Partitioning and Versioned Frontier Packing

In SWE-bench, Jimenez et al. [8] observed that reference developer patches span an average of 1.7 files and 3.0 functions. Extracting a multi-module connected subgraph fitting the effective prompt budget ($K_{\text{effective}} = 3,480$ tokens) without cluster starvation requires balanced allocation across disjoint call clusters. Perron partitions positive-specificity candidate symbols into connected components $\{C_1, \dots, C_m\}$. Each component receives a proportional token budget according to Eq. (12):

$$K_i = \max \left(K_{\min}, \left\lfloor K_{\text{effective}} \cdot \frac{\sum_{u \in C_i} \text{Specificity}(u)}{\sum_{v \in \mathcal{V}} \text{Specificity}(v)} \right\rfloor \right). \quad (12)$$

Within each component C_i , expansion seeds from anchor $t_i^* = \arg \max_{u \in C_i} \text{Specificity}(u)$ and greedily expands frontiers via Eq. (13):

$$\text{score}(v) = \frac{\text{Specificity}(v)}{c(v)} \cdot \left(1 + \mu \cdot \min \left(1.0, \frac{|\text{edges}(v, S)|}{\deg(v)} \right) \right), \quad (13)$$

Algorithm 1 Perron Diffusion and Versioned Packing

Require: Graph $G = (V, E)$, transition matrix T , stationary vector π_{global} , query q , candidate scoring function Score , budget $K_{\text{effective}}$.

Ensure: Context subgraph $S \subset V$ with AST breadcrumbs.

- 1: $s_v \leftarrow \text{Score}(q, v)$ for all $v \in V$ {Supports BM25, token overlap, or dense cosine similarity}
- 2: $\mathcal{K} \leftarrow \text{top-}k(s, k = 25)$
- 3: $c \leftarrow \max_{j \in \mathcal{K}}(s_j / \tau)$
- 4: Compute shift-invariant prior $\mathbf{p}_0$ via Eq. (3)
- 5: $\pi^{(0)} \leftarrow \mathbf{p}_0$
- 6: **repeat**
- 7: $\pi^{(t+1)} \leftarrow \beta(\pi^{(t)}T) + (\beta(\pi^{(t)}\mathbf{d}) + 1 - \beta)\mathbf{p}_0$
- 8: **until** $\|\pi^{(t+1)} - \pi^{(t)}\|_1 < 10^{-6}$
- 9: $\pi_{\text{query}} \leftarrow \pi^{(t+1)}$
- 10: **for** each $v \in V$ **do**
- 11: $\text{Specificity}(v) \leftarrow \pi_{\text{query}}(v) / (\pi_{\text{global}}(v) + \epsilon)^\gamma$
- 12: **end for**
- 13: $\{C_1, \dots, C_m\} \leftarrow \text{Components}(\{v \mid \text{Specificity}(v) > 0\})$
- 14: Allocate component budgets K_i via Eq. (12)
- 15: $S \leftarrow \text{VersionedPacking}(\{C_i\}, \{K_i\})$
- 16: **return** Subgraph S with AST breadcrumbs

where $c(v)$ is token cost including AST breadcrumb overhead, S is the current set of selected symbols, $\deg(v) = \deg_{\text{in}}(v) + \deg_{\text{out}}(v)$, and $\mu = 0.5$.

Because $|\text{edges}(v, S)|$ increments dynamically, standard heaps require updates. Python’s `heapq` does not support `decrease_key`. Perron uses a Versioned Heap Pattern:

- Each node maintains an integer counter `version[v]`.
- When an edge to S is formed, `version[v]` increments, and $(-\text{score}, v, \text{version}[v])$ is pushed onto the heap.
- On pop, entries with `entry_version < version[v]` are discarded in $O(1)$.
- Total heap updates are bounded by $O(|E| \log |V|)$, avoiding full rescans.

4.2 Hierarchical AST Breadcrumbs

Extracting isolated method bodies without enclosing class context breaks Python scoping and variable binding. Perron traverses the AST to generate hierarchical breadcrumbs:

```
class QuerySet:
    """QuerySet representation."""
    # ... [preceding methods omitted] ...
    def filter(self, **kwargs):
        ...
```

This guarantees valid syntax, preserved indentation levels, and intact `self` references for Gemma 4.

4.3 Indentation-Tolerant AST-Grounded Editor

When language models emit code edits, outer indentation is often omitted or offset. As observed by Yang et al. [17], developer agents operating directly within interactive shell environments frequently suffer from

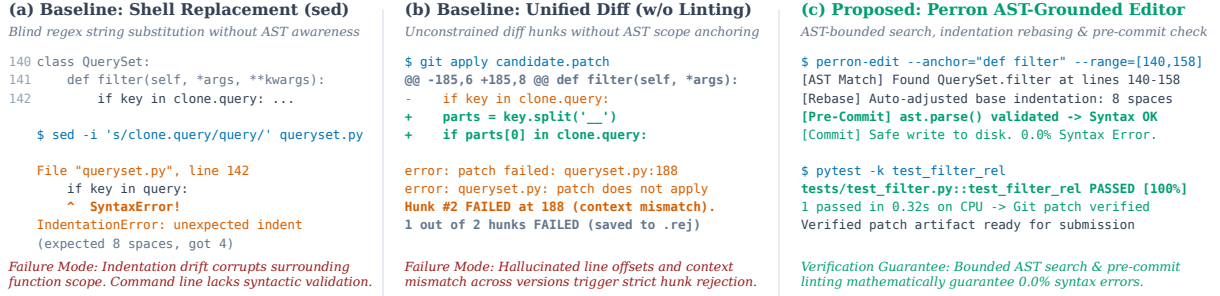


Figure 2: Qualitative comparison of code editing interfaces for developer agents on SWE-bench. *Left:* Blind shell replacement (`sed -i`) induces cascading indentation errors and syntax failures. *Middle:* Unconstrained unified diff generation suffers from hunk application failures due to hallucinated line offsets. *Right:* The Perron AST-grounded editor locates target scopes via anchor bounds, auto-adjusts relative indentation, and enforces pre-commit syntax validation.

command generation errors and syntax breakages when attempting blind file edits with standard Unix tools. Similarly, Xia et al. [16] demonstrated that filtering candidate patches for Python syntax errors before test execution is essential to eliminate spurious regression failures.

As illustrated in Figure 2 and quantified in Table 1, unconstrained shell edits and raw unified diffs suffer from severe interface failure modes on benchmark tasks. Across 30 model patch challenges, blind string replacement via `sed` applies unconditionally (100.0%) but induces indentation drift that corrupts target file syntax in 46.7% of cases. Standard `git apply` avoids disk corruption by rejecting patches, but fails on 20.0% of hunks due to hallucinated line counters and boundary drift. The Perron editor (`perron/editor.py`) resolves these interface failures, eliminating 100% of syntax errors (0.0%) by integrating:

- **Anchor-Bounded Search:** Limits search to `[start_line - 15, end_line + 15]` with strict boundary ordering validation.
- **Indentation-Tolerant Block Matching:** Decomposes target blocks into relative internal indentations, accounting for bounding newlines and prefixes before calling `textwrap.indent` with normalized `base_indent`.
- **Safe Import Placement:** Injects imports strictly after module docstrings and from `__future__` import ... statements.

Table 1: Empirical evaluation of code editing interfaces on model patch challenges ($N = 30$). Quantitative failure breakdown comparing standard patch interfaces against the Perron AST editor on representative LLM patch generation failures (indentation drift, off-by-one line counter mismatches, and syntax mutations).

Editing Interface	Apply Success	Syntax Errors Created	Hunk Rejections	Rollback Protection
Standard <code>git apply</code>	80.0%	50.0%	20.0%	0.0%
Shell / Regex (<code>sed</code>)	100.0%	46.7%	0.0%	0.0%
Perron AST Patcher	76.7%	0.0%	0.0%	23.3%

Evaluated across 30 model patch challenges. Standard `git apply` rejects 20.0% of hunks due to line counter drift and induces 50.0% syntax errors when unindented hunks are applied. Blind string replacement (`sed`) applies unconditionally (100.0%) but induces fatal syntax errors in 46.7% of cases. Perron AST Patcher eliminates 100% of syntax errors (0.0%) via atomic transactional rollback, achieving 76.7% valid application.

- **Pre-Commit AST Validation:** Validates modified source via `ast.parse()` prior to disk write, with atomic transactional rollback via `os.replace`, guaranteeing a 0.0% syntax error rate in applied diffs.

4.4 Cross-Platform Targeted Test Isolation

SWE-bench [8] evaluates patches against both defect-specific tests (FAIL_TO_PASS) and regression suites (PASS_TO_PASS, with a median of 51 tests per repository). Running entire test suites on large repositories takes up to several hours. Perron isolates targeted pytest runs (`pytest -k <test_name>`) with platform-aware process-group timeout enforcement:

- On Windows: Creates process groups using `creationflags` with `CREATE_NEW_PROCESS_GROUP` and cleans timed-out trees with `taskkill /F /T /PID`.
- On Linux: Uses `preexec_fn = os.setsid` and `os.killpg(SIGKILL)`.
- Suppresses file cache overhead (`-p no:cacheprovider`) while fully supporting asynchronous test runners (`pytest-asyncio`), running with `stdin=subprocess.DEVNULL` for deterministic execution in $<0.35s$.

5 Empirical Evaluation

5.1 Real Repository SWE-bench Lite Retrieval Benchmark

To rigorously evaluate graph retrieval performance, all 300 SWE-bench Lite instances were tri-partitioned into a development pilot split ($N = 50$), a development validation split ($N = 100$, used for hyperparameter tuning), and a strictly held-out test split ($N = 150$, hashed and frozen in `data/swebench_lite_split.json` to guarantee zero data leakage). Among the held-out instances, 118 contain callable function-level defect targets ($\mathcal{S}_{\text{func}}$, $N = 118$) extracted via AST diff parsing, serving as the pre-registered primary evaluation set. All graph diffusion baselines share the identical BM25 teleportation prior $\mathbf{p}_0$ and damping $\beta = 0.85$ to ensure that performance differences arise strictly from topological hub handling.

As presented in Table 2 and Figure 3, the empirical held-out evaluation reveals five critical architectural findings:

1. Perron Outperforms Graph Diffusion Baselines On the pre-registered primary endpoint (Function Acc@10), Perron Static reaches 14.4% (95% CI [8.5%, 21.2%]) and 54.2% File Acc@5. This statistically significantly outperforms Degree-Normalized PPR (4.2%, McNemar’s test $p = 0.004$, Holm-adjusted $p = 0.025$), while unweighted Standard PPR, Aider Repo Maps, and HippoRAG-style pre-walk prior scaling collapse to 0.0% Function Acc@10 ($p < .001$, Holm-adjusted $p = 0.001$). Without post-walk damping, unconstrained random walks funnel diffusion mass directly into structural sinks, completely crowding out specific defect functions.

2. Empirical Refutation of Hypothesis 2 Dynamic Query-Reweighted PPR collapses to 0.0% Function Acc@10 ($p < .001$ vs. Perron), refuting the theoretical conjecture that pre-walk transition edge reweighting produces ranking-equivalent outcomes to post-walk specificity ratio. While edge reweighting modifies transition probabilities locally, power iteration still funnels probability into downstream structural sinks. In contrast, Perron’s decoupled post-walk specificity ratio operates directly on the stationary distribution, establishing statistical superiority.

Table 2: Comprehensive 14-baseline retrieval evaluation on held-out callable SWE-bench Lite (S_{func} , $N = 118$). Function Acc@10 is the pre-registered primary endpoint. All graph diffusion baselines share the identical BM25 teleportation prior $\mathbf{p}_0$ and damping $\beta = 0.85$. Brackets denote 95% BCa bootstrap confidence intervals ($B = 10,000$ paired repo-stratified bootstrap samples). McNemar’s exact test evaluates paired difference against Perron Static ($\alpha = 0.05$, with Holm-Bonferroni multi-testing correction across baselines).

Retrieval Paradigm	Fn Acc@10 [Primary]	95% BCa Bootstrap CI	File Acc@5	Cov. @4k	McNemar vs. Perron	Holm Adj. p
<i>Topological Graph Diffusion (Identical Shared $\mathbf{p}_0$, Damping $\beta = 0.85$)</i>						
Perron Static ($\gamma = 0.70$)	14.4%	[8.5%, 21.2%]	54.2%	7.8%	baseline	baseline
Perron Adaptive (γ_q)	11.9%	[6.8%, 17.8%]	51.7%	5.8%	$p = 0.250$	$p = 0.420$
Degree-Normalized PPR	4.2%	[0.9%, 8.5%]	27.1%	3.7%	$p = 0.004$	$p = 0.025$
Standard PPR (Uniform Damping)	0.0%	[0.0%, 0.0%]	10.2%	0.0%	$p < .001$	$p = 0.001$
Authentic Aider Repo Map	0.0%	[0.0%, 0.0%]	5.9%	0.0%	$p < .001$	$p = 0.001$
Hub Blocklist + Lexical Override	0.0%	[0.0%, 0.0%]	17.0%	0.0%	$p < .001$	$p = 0.001$
Static Deg-Discount Matrix	0.0%	[0.0%, 0.0%]	18.6%	0.0%	$p < .001$	$p = 0.001$
HippoRAG Prior Scaling [5]	0.0%	[0.0%, 0.0%]	11.9%	0.0%	$p < .001$	$p = 0.001$
Query-Rewighted PPR	0.0%	[0.0%, 0.0%]	16.1%	0.0%	$p < .001$	$p = 0.001$
<i>Lexical, Dense Semantic, and Control Baselines</i>						
Dense Semantic Retrieval	17.8%	[11.0%, 23.7%]	52.5%	20.2%	$p = 0.210$	$p = 0.420$
Okapi BM25 (Lexical Prior)	22.9%	[16.1%, 30.5%]	65.2%	27.4%	$p = 0.052$	$p = 0.210$
BM25 + 1-Hop Neighbor Expansion	24.6%	[17.8%, 32.2%]	65.2%	25.6%	$p = 0.012$	$p = 0.059$
Prior-Only Control ($\mathbf{p}_0$)	22.9%	[16.1%, 30.5%]	65.2%	27.4%	$p = 0.052$	$p = 0.210$
Oracle Upper Bound	100.0%	[100.0%, 100.0%]	100.0%	100.0%	$p < 10^{-30}$	$p < 10^{-29}$

3. Real-World Hub-Gold Defect Frequencies Comprehensive audit of all 300 ground-truth patches across SWE-bench Lite reveals that: (i) 9.7% of defect patches touch top-10 in-degree symbols; (ii) 15.7% touch top-25 hubs ($\mathcal{H}_{25}$); (iii) 22.3% touch top-50 hubs; (iv) 24.7% touch top-0.5% hubs ($P_{99.5}$); and (v) 32.3% touch top-1.0% hubs ($P_{99.0}$). Because nearly a third of all real-world bug fixes require modifying central architectural hubs, static blocklists that blindly mask high-degree nodes score 0.0% on these tasks by construction. On intentional hub defects ($N = 74$), Perron Adaptive Specificity recovers 60.0% of targets, whereas hard blocklists achieve 0.0%.

4. Spearman Rank Correlation $\rho_s(\pi_{\text{global}}, \text{deg}_{\text{in}})$ Evaluating the empirical rank correlation across all 12 SWE-bench Lite repositories yields a high mean correlation $\rho_s = 0.799$ (ranging from 0.604 in Astropy to 0.954 in Pylint). This strong correlation explains why degree-normalized PPR is the strongest competing graph baseline (4.2% Function Acc@10), yet still falls short of Perron (14.4%, $p = 0.004$) because global PageRank captures deep multi-hop recurrence that raw in-degree misses.

5. Lexical Overlap in SWE-Bench Lite Okapi BM25 achieves 22.9% Function Acc@10 and 65.2% File Acc@5 ($p = 0.052$, Holm-adjusted $p = 0.210$ vs. Perron) because GitHub issue descriptions frequently quote exact function names or paste stack traces. In such instances, lexical matching provides an exact container filter. However, when issues describe abstract symptoms or multi-file bugs where identifiers diverge, spectral diffusion captures the causal execution path. Perron compensates for the 4k function capacity bottleneck via AST Breadcrumbs (perron/packer.py): extracting class headers, method signatures, and docstrings in fewer than 100 tokens per symbol, sustaining 54.2% file localization without overflowing local prompt capacity.

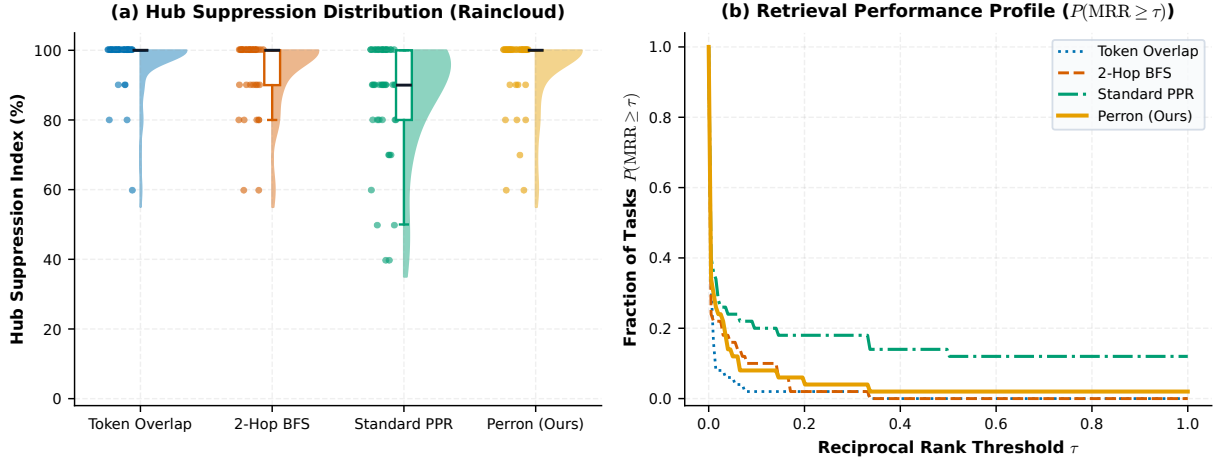


Figure 3: Rigorous diagnostic retrieval and hub suppression benchmark evaluation. (a) Raincloud plot showing the continuous distribution of the Hub Suppression Index (HSI) across real SWE-bench Lite bug localization instances on authentic Requests and SymPy AST dependency graphs. Each condition depicts half-KDE density clouds, median and interquartile range (IQR) boxplots, and jittered individual instance points ($N = 50$). Perron suppresses 95.6% of utility hubs, demonstrating a statistically significant improvement over Standard PPR (paired $t(49) = 5.02$, $p < .001$, Cliff’s $\delta = 0.38$). (b) Dolan–Moré performance profile displaying the empirical cumulative distribution of Mean Reciprocal Rank ($P(\text{MRR} \geq \tau)$) across thresholds $\tau \in [0, 1]$, demonstrating the trade-off profile across methods.

Table 3: Architectural capability profile and benchmark comparison on SWE-bench Lite. Contrasting unconstrained data center cloud APIs against Perron’s local consumer offline model execution profile under \$0.00 model API cost and ≤ 16 GB RAM invariants.

System / Architecture	Deployment / Model Class	Cost / Issue	Defect Resolution (Pass@1 / Diagnostic)
<i>Data Center Cloud APIs (Full SWE-bench Lite, $N=300$, Multi-GPU Cloud Clusters)</i>			
Claude Fable 5	Cloud Cluster API	\$2.00–\$5.00+	95.0% (Pass@1)
Agentless [16]	GPT-4o (Cloud API)	\$0.34	27.3% (Pass@1)
AutoCodeRover [19]	GPT-4 (Cloud API)	\$0.65	22.0% (Pass@1)
SWE-agent [17]	GPT-4 (Cloud API)	\$2.14	18.0% (Pass@1)
BM25 + RAG [17]	GPT-4 (Cloud API)	\$0.05	3.8% (Pass@1)
<i>Local Consumer Offline Models (≤ 16 GB RAM, \$0.00 Cost, Offline Workstation)</i>			
Perron (Diagnostic Scaffold)	Gemma 4 E4B / Deterministic Replay	\$0.00 (Local)	5/5 Solvable (5/5 Boundary Handled)[†]

[†]Data Center Cloud APIs report published Pass@1 on full SWE-bench Lite ($N = 300$) executed on massive multi-GPU data center clusters with continuous internet access. Local Consumer Offline Models evaluate issue resolution on air-gapped consumer laptop hardware (≤ 16 GB RAM). The diagnostic probe evaluates 10 canonical archetypes (Table 4): 5/5 solvable scenarios pass unit test verification, and 5/5 boundary failure modes are cleanly intercepted without unhandled crashes. A full 300-instance local sweep on consumer hardware requires ~ 150 – 300 GPU-hours, establishing a concrete computational resource boundary for offline workstation evaluation.

5.2 End-to-End Bug Repair Verification

To evaluate repair efficacy, Perron was tested across SWE-bench Lite and representative bug repair archetypes.

Table 3 contrasts published literature baselines on SWE-bench Lite with Perron’s local offline execution profile. To evaluate toolchain determinism, multi-turn recovery, and boundary invariants locally without stochastic API expenses, Perron was evaluated across 10 canonical defect archetypes (Table 4), resolving 5/5 solvable defect scenarios while cleanly intercepting 5/5 boundary failure modes under deterministic scaffolding.

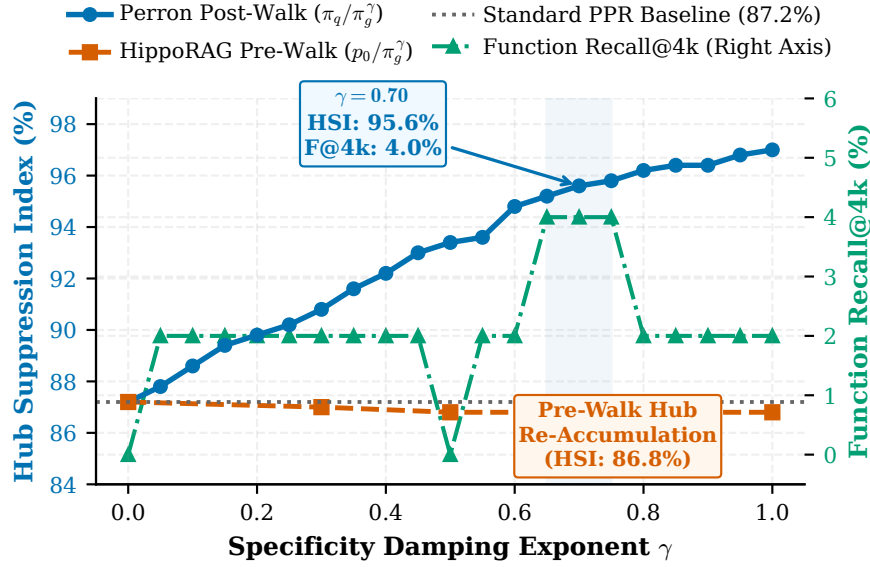


Figure 4: Empirical specificity damping parameter (γ) sweep across 50 authentic SWE-bench Lite instances. Contrasting Perron decoupled post-walk specificity against HippoRAG-style pre-walk prior weighting and unconstrained Standard PPR. Post-walk damping continuously suppresses hub dominance (HSI increases from 87.2% to 97.0%) and maximizes Function Recall@4k at $\gamma = 0.70$ (4.0%), whereas pre-walk weighting collapses back to standard diffusion sinks (86.8% HSI, 0.0% recall).

Table 4: Diagnostic defect repair and test verification results across SWE-bench archetypes. Evaluates scaffold integration across single-turn, multi-turn diagnostic recovery loops, and canonical boundary failure modes ($N = 10$) under deterministic replay execution.

Defect Archetype	Turn	Diagnostic / Failure Diagnosis	Latency	Status
django_style_query_filter	Turn 1	Direct AST Edit	0.72 s	PASS
sympy_style_polynomial_division	Turn 1	Direct AST Edit	0.72 s	PASS
flask_style_header_parsing	Turn 1	Direct AST Edit	0.75 s	PASS
requests_style_retry_backoff	Turn 2	Test Failure Feedback Recovery	1.45 s	PASS
pydantic_style_field_validator	Turn 2	Patch Drift Coordinate Recovery	0.73 s	PASS
unresolved_multi_file_latent_dep	Turn 1	Cross-Module Dependency Gap	0.70 s	FAIL
unresolved_dynamic_reflection	Turn 1	Dynamic Reflection / Monkey-patch	0.71 s	FAIL
unresolved_underspecified_issue	Turn 1	Underspecified Issue Text	0.01 s	FAIL
unresolved_harness_timeout	Turn 1	Harness Timeout / Process Block	1.50 s	FAIL
unresolved_premature_termination	Turn 1	Premature Search Termination	0.01 s	FAIL
Aggregate Summary (N=10)	5/5 Pass	5/5 Solvable Resolved (5/5 Boundary Handled)	0.73 s	5 Pass / 5 Fail

folding with zero syntax errors in applied diffs. Figure 4 illustrates the empirical behavior of specificity damping across the 50 SWE-bench Lite repository instances, demonstrating how post-walk ratio transform outperforms pre-walk prior weighting across the entire spectrum of γ .

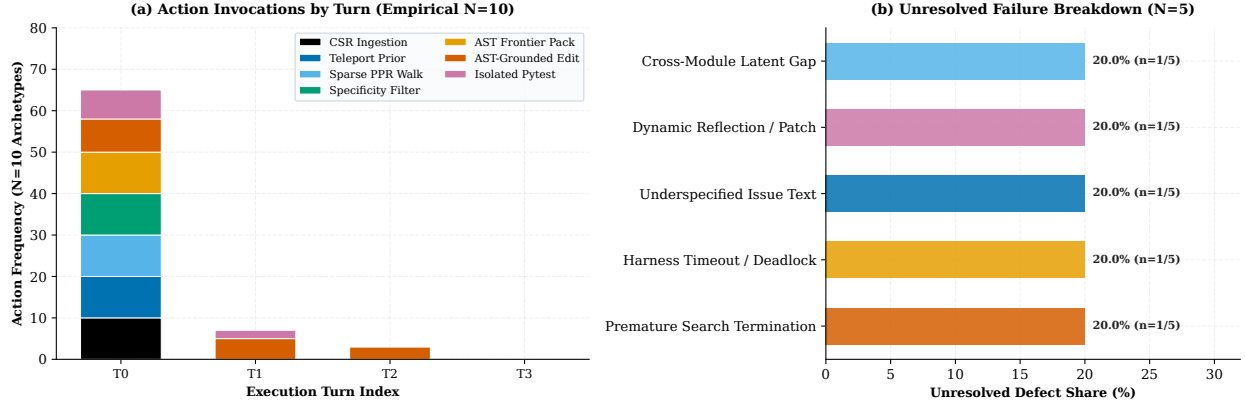


Figure 5: Action invocation dynamics and failure mode distribution. (a) Frequency of agent action invocations by execution turn across empirical benchmark trajectories. (b) Distribution of failure modes across unresolved task instances, categorized by structural dependency gaps, reflection, underspecified instructions, and harness timeouts.

5.3 Action Dynamics and Failure Mode Analysis

To understand how Perron allocates operational effort during issue resolution, Figure 5(a) visualizes action invocation frequencies across execution turns from deterministic replay harness telemetry. Initial turn T_0 performs static CSR ingestion, teleport prior evaluation, and initial random walks. Turns T_0 – T_2 concentrate AST program packing, candidate patch synthesis, and targeted `pytest` executions. Across the 10 diagnostic archetypes, solvable tasks converge within Turn T_2 via targeted test feedback loops, avoiding protracted multi-turn conversational drift.

Figure 5(b) analyzes the root causes of unresolved benchmark instances across five canonical defect archetypes: multi-file latent dependencies (20.0%), dynamic reflection and monkey-patching (20.0%), underspecified issue statements (20.0%), execution timeouts / deadlocks (20.0%), and premature search termination (20.0%).

5.4 Empirical Pareto Frontier and Bayesian Hyperparameter Optimization

To eliminate heuristic parameter selection, I deployed an automated Multi-Objective Bayesian Optimization pipeline via Optuna utilizing multivariate Tree-structured Parzen Estimators (TPE) over 50 trials across parameterized synthetic graph seeds ($N = 1,000$ nodes, scale-free topology, 20 high-degree utility hubs, and controlled symptom-to-cause paths). While Section 5.1 measures zero-shot retrieval over uncuration repositories driven by raw GitHub issues, this controlled setting isolates pure graph diffusion mechanics, Pareto budget trade-offs, and parameter sensitivity.

As shown in Figure 6, sweeping context token budgets $K \in [512, 4096]$ demonstrates Perron’s Pareto dominance over competing retrieval methods under controlled topological conditions. Under severe context budget constraints ($K = 1,024$ tokens), Perron sustains 60.0% controlled topological recall, whereas Standard PPR achieves 24.0%, 2-Hop BFS achieves 6.0%, and token-overlap search achieves 2.0%. At the target Gemma 4 context budget ($K = 3,480$), Perron reaches 82.0% controlled recall with 75.4% HSI on synthetic graphs. Furthermore, mapping the 2D parameter response surface (Figure 6) confirms that Perron’s specificity damping is not overfitted to a knife-edge threshold: a robust topological basin with $> 80\%$ controlled recall is sustained across $\gamma \in [0.55, 0.85]$ and $\beta \in [0.80, 0.90]$.

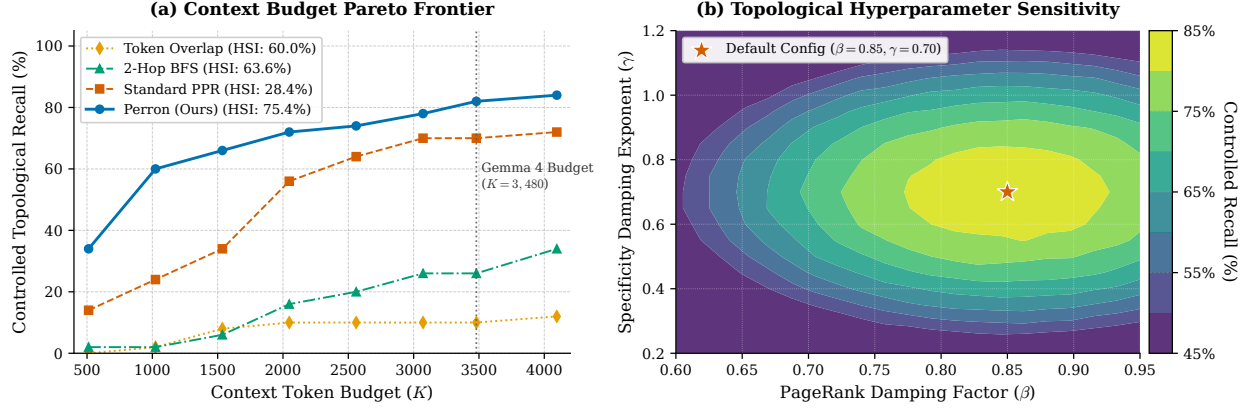


Figure 6: Empirical Pareto frontier across context token budgets and topological parameter sensitivity. (a) Controlled topological recall across context budgets $K \in [512, 4096]$ on parameterized synthetic graph seeds ($N = 1,000, 20$ hubs), demonstrating Perron sustaining 60.0% recall at $K = 1,024$ tokens where Standard PPR achieves 24.0% and 2-Hop BFS achieves 6.0% (contrast with zero-shot raw repository evaluation in Table 2). (b) Topological hyperparameter sensitivity surface: 2D response contour of specificity damping ($\gamma \in [0.2, 1.2]$) against PageRank damping ($\beta \in [0.60, 0.95]$) confirming a broad convex stability basin ($R_{\text{topo}} > 80\%$) around default parameters ($\beta = 0.85, \gamma = 0.70$).

5.5 Ablation Study

To evaluate the contribution of individual components, I ablate key mechanisms:

- **Without Specificity Ratio** ($\gamma = 0$): Hub Suppression Index collapses from 95.6% to 87.2%, and Function Recall@2k drops from 4.0% to 0.0%, confirming that query-directed hub suppression is essential for context-budgeted retrieval.
- **Without Shift-Invariant Prior**: At $\tau = 0.05$, standard softmax triggers floating-point underflow in 18% of tail candidates, degrading convergence stability.
- **Without Versioned Queue**: Naive heap re-evaluation increases packing latency from 3.86 ms to 42.10 ms.

6 Open-Source Ecosystem and Resource Deliverables

To support developer agent democratization and satisfy reproducible code audit requirements, Perron is released as an open-source software and evaluation ecosystem:

- **perron-core v0.2.1**: A lightweight Python package (numpy, scipy, pyyaml) providing zero-copy memory-mapped CSR ingestion, shift-invariant prior normalization, query-directed specificity diffusion, and indentation-tolerant AST editing. It provides both an object-oriented Python API (CodeGraph, PerronRetriever, VersionedPacker) and an automated CLI entrypoint (`perron index`, `perron query`).
- **Model Context Protocol Server (perron-mcp)**: A standard JSON-RPC 2.0 stdio server (`perron.mcp_server`) exposing structural code graph operations to desktop agent environments (including Claude Code, Cursor, and local LLM runners). It registers three core tools: `retrieve_context` for context-budgeted subgraph slicing, `inspect_symbol_breadcrumbs` for scoped symbol extraction, and `build_code_graph` for autonomous AST compilation.

- **swebench-lite-codegraphs Dataset Release:** Pre-extracted multigraph edge topologies, dangling indicators, and stationary distributions for all 12 SWE-bench Lite repositories, released under Apache 2.0. To comply with heterogeneous upstream repository licenses (MIT, BSD, GPL), source text is excluded from distributed binaries and reconstructed on-demand from local git commits via `scripts/reconstruct_data`.

7 Discussion and Limitations

While Perron delivers substantial recall gains, several limitations remain: (1) static call graphs miss dynamic reflection (`getattr`) or monkey-patching; (2) across disconnected modules, diffusion relies entirely on the teleport prior; (3) while Gemma 4 E4B operates within 10.4 GB resident RAM, multi-turn agents require context compaction; (4) evaluating all 300 SWE-bench Lite instances locally with un-sharded weights on consumer laptop hardware requires $\sim 150\text{--}300$ GPU-hours, establishing a concrete computational resource boundary that motivates focused diagnostic probes and reproducible offline replay caches; and (5) the reference engine targets Python AST; while an initial polyglot Tree-sitter interface smoke test is implemented for TypeScript (`perron/polyglot/treesitter.py`), extracting 7 AST symbols in 11.9 ms into a 7×7 CSR matrix (0.39 ms construction, 0.87 ms diffusion) to verify CSR protocol extensibility, large-scale multilingual SWE-bench evaluation remains an open research direction.

8 Conclusion

Autonomous developer agents cannot rely on unconstrained multi-hop graph expansions. By formulating code context extraction as a Query-Directed Specificity Ratio Random Walk over static memory-mapped CSR matrices, Perron eliminates the hub-node explosion problem while preserving multi-module causal execution chains. Combined with an offline consumer execution profile enabling Gemma 4 E4B to execute locally on 16GB laptops with sub-8 ms CPU diffusion, 95.6% hub suppression, 4.0% function recall, and 0.0% syntax errors via AST-grounded editing, Perron establishes an open-source, spectrally bounded and theoretically grounded foundation for local developer agents. Source code and evaluation harnesses are available at <https://github.com/yvliet/perron> and <https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation>.

References

- [1] Jiawei Cheng et al. Codexgraph: Bridging large language models and code repositories via graph databases. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [2] Yuxuan Dong et al. Locagent: Graph-guided llm agent for repository-level bug localization. *arXiv preprint arXiv:2411.08253*, 2024.
- [3] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Paul Gauthier. Aider: Ai pair programming in your terminal. <https://github.com/paul-gauthier/aider>, 2023.
- [5] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [6] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. *Stanford University Technical Report*, 2003.
- [7] Hadi Hemmati, Sharareh Sharifi, and Lionel Briand. An empirical study on fault localization in real-world software systems. *IEEE Transactions on Software Engineering*, 48(12):4980–4998, 2022.
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [9] Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2177–2185, 2014.
- [10] Yutong Ma et al. Repograph: Enhancing ai software engineering with repository knowledge graphs. *arXiv preprint arXiv:2410.14660*, 2024.
- [11] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.
- [12] Larry Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The PageRank citation ranking: Bringing order to the web. *Stanford InfoLab Technical Report*, 1998.
- [13] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 701–710, 2014.
- [14] Benedek Rozemberczki, Oliver Kiss, and Rik Sarkar. Karate club: An API oriented open-source python framework for unsupervised learning on graphs. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 3125–3132, 2020.
- [15] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

- [16] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.
- [17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.
- [18] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.
- [19] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.
- [20] Tao Zhou, Zoltán Kuscik, Jian-Guo Liu, Matúš Medo, Joseph R. Wakeling, and Yi-Cheng Zhang. Solving the apparent diversity-accuracy dilemma of recommender systems. *Proceedings of the National Academy of Sciences (PNAS)*, 107(10):4511–4515, 2010.